

Lab 1: Requirements Engineering & UML Use-Case Modelling

Problem Statement #23 – Hyperlocal Courier Dispatch & Tracking Engine

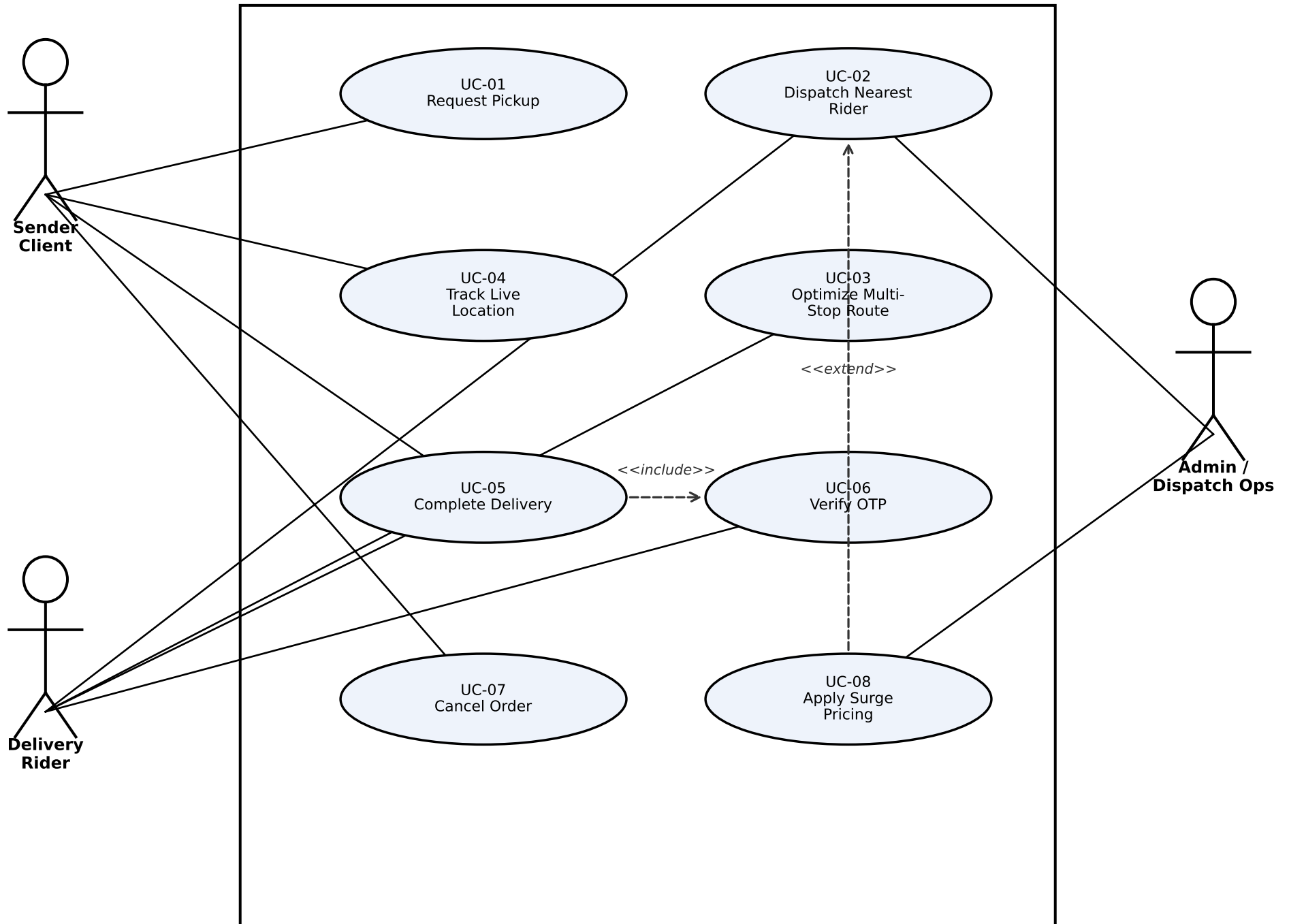
Name: Mathin M SRN: PES1UG24AM377 Section: G

Requirements Table

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall match an outgoing courier request with the nearest active delivery rider within a 3 km radius.	High	Pass: Rider receives dispatch notification with pickup location. Fail: Order assigned to offline rider.	Core matching function of the platform, given in problem statement.
FR-002	Functional	The system shall verify delivery completion by requiring the rider to enter a 4-digit OTP shared with the sender at drop-off.	High	Pass: order status changes to “Delivered” only when entered OTP matches the one generated for that order. Fail: status updates to Delivered without OTP match / with wrong OTP.	Prevents fraudulent delivery marking and confirms parcel reached the right person.
FR-003	Functional	The system shall generate an optimized stop sequence for a rider who has been assigned more than one pickup or drop in the same run.	Medium	Pass: for a rider with ≥ 2 stops, system output route has total distance $\leq$ the distance of the original (unsorted) stop order. Fail: stops kept in the order they were received.	Reduces delivery time and fuel/battery cost for riders handling multiple parcels.
FR-004	Functional	The system shall let the sender client view the assigned rider's live location on a map from pickup confirmation until delivery.	High	Pass: rider marker position on sender's app refreshes at an interval of 5 seconds or less. Fail: marker does not update or updates after >5 sec.	Directly improves customer trust and transparency of an on-demand delivery service.
FR-005	Functional	The system shall allow the sender client to cancel a request any time before the rider marks the pickup as collected.	Medium	Pass: Cancel option is available and functional until pickup is confirmed by rider; order status changes to “Cancelled”. Fail: cancellation still possible after pickup is confirmed.	Gives sender flexibility to back out of a request without penalising a rider already carrying the parcel.
NFR-001	Performance & Security	Delivery status and live GPS telemetry updates from riders shall transmit to the sender client with under 2-second latency.	High	Pass: Benchmarking tests confirm target latency and security standards under simulated peak load.	Given requirement — real time tracking is the core value proposition of the app.
NFR-002	Reliability	If a dispatched rider does not accept an assigned order within 30 seconds, the system shall automatically reassign the order to the next-nearest available rider.	High	Pass: in $\geq 95\%$ of simulated timeout cases the order is reassigned within 30 sec of the original rider not responding. Fail: order stays unassigned/stuck after timeout.	Avoids orders getting stuck because one rider is unresponsive; keeps overall dispatch loop reliable.

Actors identified: Sender Client, Delivery Rider, Admin / Dispatch Ops.

Hyperlocal Courier Dispatch & Tracking Engine



Use-Case Flow Specification

Problem Statement #23 – Hyperlocal Courier Dispatch & Tracking Engine

Name: Mathin M SRN: PES1UG24AM377 Section: G

Use Case: UC-05 – Complete Delivery

Primary Actor: Delivery Rider

Secondary Actor: Sender Client

Preconditions

1. The rider has accepted the assigned order and is at the sender's location or the delivery destination.
2. The order status in the system is “In Transit” and an OTP has already been generated and shared with the sender for this order.

Postconditions

Success: Order status changes to “Delivered”, delivery timestamp is logged, and both sender and rider apps show the updated status.

Failure: Order remains “In Transit” and an OTP-mismatch flag is logged against the order.

Main Success Scenario

1. Rider reaches the drop-off location and taps “Mark as Delivered” on the app.
2. System prompts the rider to enter the OTP provided by the recipient.
3. Rider asks the recipient for the OTP and enters it into the app.
4. System (invoking <<include>> UC-06 Verify OTP) checks the entered OTP against the one generated for this order.
5. OTP matches; system marks the order status as “Delivered” and timestamps it.
6. System notifies the sender client that the parcel has been delivered.
7. System closes the order and makes the rider available for the next dispatch.

Alternate Flow – 4a: OTP Does Not Match

- 4a1. System displays an “Incorrect OTP” message to the rider and does not change the order status.
- 4a2. Rider is prompted to re-confirm the OTP with the recipient and re-enter it.
- 4a3. If the OTP fails a second time, the system flags the order for Admin / Dispatch Ops review and keeps the status as “In Transit” instead of auto-completing it.
- 4a4. Use case resumes at step 2 once a correct OTP is entered, or ends (flagged) if the rider exits the screen.